

Assignment 3 | Bridge Pattern

Timur Naumov | SE-2537

Topic D - Remote controls

Repository: <https://github.com/tnaumovv/Assignment3-Bridge-Pattern>

Submitted source commit: d3cebeb7c01d13df56d3f86a8da1c7617453a00a

Design and UML

The two choices are remote behavior and device type. BasicRemote powers a device on at volume 30; QuietRemote uses 5. Remote holds a Device reference received through its constructor, so either remote can use any device without a subclass for each combination. This is the Bridge link [1, 3].

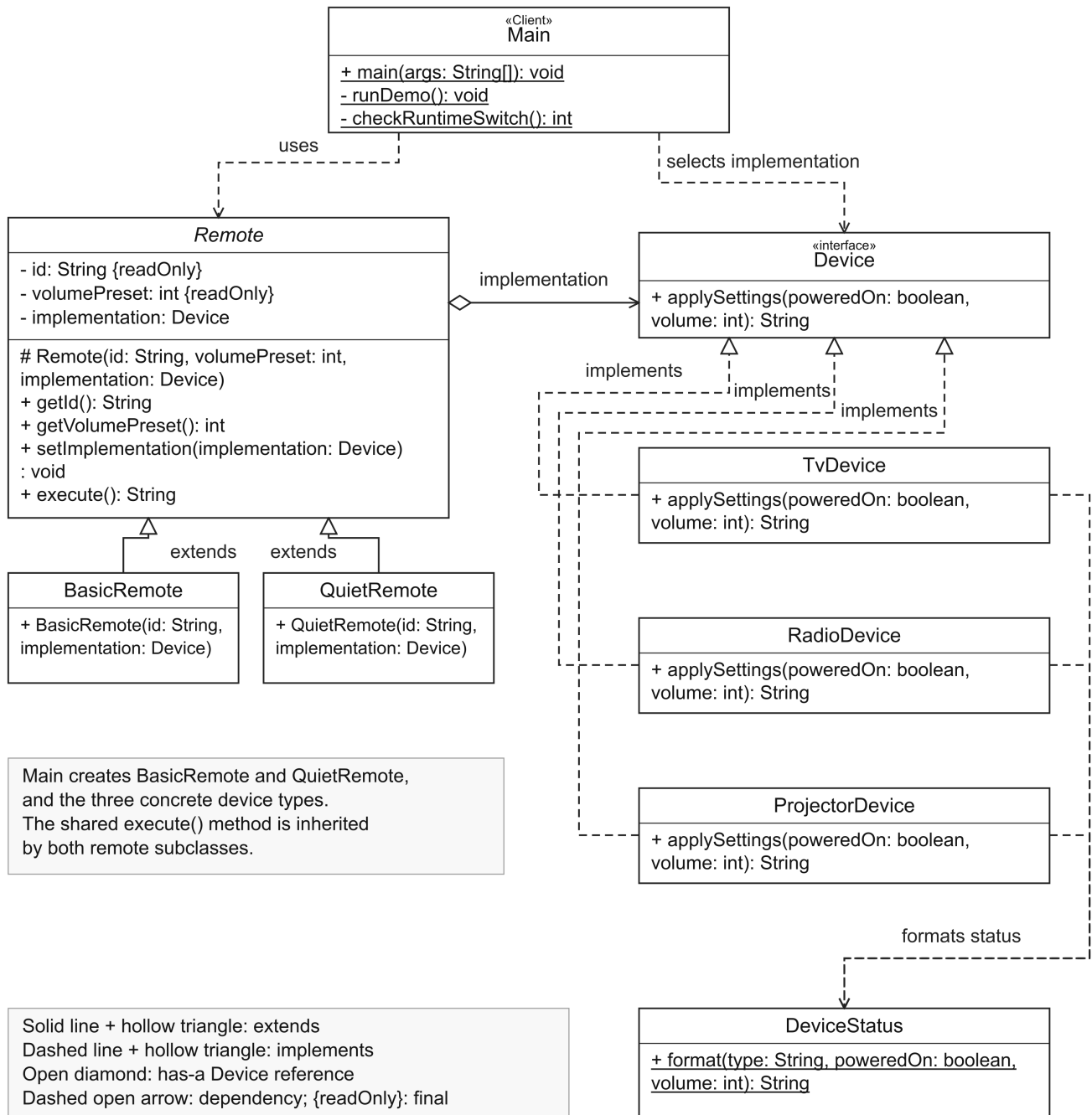


Figure 1. Class diagram of the submitted code.

Code examples

Code example	Practice and benefit
<i>Remote.java</i> <pre>public String execute() { return implementation.applySettings(true, volumePreset); }</pre> <i>TvDevice.java</i> <pre>public String applySettings(boolean poweredOn, int volume) { return DeviceStatus.format("TV", poweredOn, volume); }</pre>	Separation of responsibilities Remote supplies settings through Device. The device returns its type and status. Remote does not name a concrete device.
<i>Main.java - checkRuntimeSwitch()</i> <pre>boolean sameObject = original == switched; boolean stateUnchanged = originalId.equals(switched.getId()) && originalVolume == switched.getVolumePreset();</pre>	Meaningful names sameObject checks reference identity. stateUnchanged checks the saved ID and volume preset. The names state what is tested.
<i>Remote.java</i> <pre>public void setImplementation(Device implementation) { this.implementation = implementation; }</pre>	Small focused methods The setter replaces one reference. It does not recreate the remote or change its settings.
<i>DeviceStatus.java - format()</i> <pre>String power = poweredOn ? "ON" : "OFF"; return type + " power=" + power + " volume=" + volume;</pre>	Avoiding duplication All three devices reuse this format. Changing the status text requires one change rather than three.
<i>Remote.java</i> <pre>private final String id; private final int volumePreset; private Device implementation; public int getVolumePreset() { return volumePreset; }</pre>	Encapsulation of state ID and volumePreset are private and final. The getter gives read access; only the Device reference can be replaced.

BasicRemote and QuietRemote set their presets with super(id, 30, implementation) and super(id, 5, implementation). Both inherit execute(), so the power-on workflow is written once.

Demonstration and extension

T1-T7 compare actual results with expected strings. T5 also uses `==` to prove the same remote object and checks its ID and volume preset before and after replacing TV with Radio.

JDK 17 - build and demo

```
$ javac --release 17 -encoding UTF-8 -d out "@sources.txt"
$ java -cp out Main --demo
```

```
T1 PASS | BasicRemote + TvDevice | result=TV power=ON volume=30
T2 PASS | BasicRemote + RadioDevice | result=RADIO power=ON volume=30
T3 PASS | QuietRemote + TvDevice | result=TV power=ON volume=5
T4 PASS | QuietRemote + RadioDevice | result=RADIO power=ON volume=5
T5 PASS | BasicRemote + TvDevice -> RadioDevice
      sameObject=true | stateUnchanged=true
      id=remote-switch -> remote-switch | volumePreset=30 -> 30
      before=TV power=ON volume=30 | after=RADIO power=ON volume=30
T6 PASS | BasicRemote + ProjectorDevice | result=PROJECTOR power=ON volume=30
T7 PASS | QuietRemote + ProjectorDevice | result=PROJECTOR power=ON volume=5
SUMMARY: 7/7 PASS
```

Figure 2. Screenshot of the recorded build and demo output on JDK 17.

The working TV/Radio version was committed first. ProjectorDevice was added afterwards. Only Main and the new ProjectorDevice changed within `src/`. Remote, both subclasses, Device, TvDevice, RadioDevice and DeviceStatus stayed unchanged.

Base commit: 544357acd1ba9ec025a578b9c9b8cb821a28c93a

Extension changes within `src/`

```
$ git diff --name-status 544357a HEAD -- src
```

```
M      src/Main.java
A      src/ProjectorDevice.java
```

Figure 3. Screenshot of the source changes recorded in `extension.diff`.

Trade-off and Adapter

Bridge separates remote behavior from device implementation. Adapter would wrap an incompatible existing device API to make it support Device [2]. The trade-off here is the extra interface and delegation for a small simulation.

References

- [1] A. Shalloway and J. R. Trott, *Design Patterns Explained*, Addison-Wesley, 2001. [Introducing the Bridge Pattern](#) (online excerpt).
- [2] Same authors and book. [Introducing the Adapter Pattern](#) (online excerpt).
- [3] Y. Bekenov, *Assignment 3 | Bridge Pattern*, Astana IT University, 2026-2027. Provided instructions.